Advanced Python Programming: Control Flow, Functions, Modules, and OOP

A Comprehensive Guide for Machine Learning and AI Freshers

Introduction

Welcome to the second module of your Python programming journey! Having mastered Python fundamentals in the previous section, you're now ready to explore the powerful features that make Python the language of choice for machine learning engineers and data scientists. This module covers **control flow**, **functions**, **modules**, **file handling**, and **object-oriented programming (OOP)**—all essential skills for building robust, scalable AI applications.

The topics you'll learn here form the backbone of professional Python development. Whether you're building machine learning pipelines, processing large datasets, or developing production-grade AI systems, mastering these concepts will significantly accelerate your career progression as an AI engineer. This guide is designed specifically for final-year graduates entering the AI/ML industry, with practical examples and best practices that you'll use daily in your professional work.

Part 1: Control Flow & Conditional Statements

Understanding Program Flow

In any programming language, **control flow** determines the sequence in which statements are executed. Python provides several mechanisms to control this flow: conditional statements for decision-making, and loops for repetition. Understanding these constructs is fundamental to writing effective programs.

Operators in Python

Before diving into conditional statements, let's understand the three types of operators used to build conditions:

Arithmetic Operators

Arithmetic operators perform mathematical operations. While primarily used for calculations, they're often combined with comparison operators in conditional statements:

Operator	Description	Example
`+`	Addition	`a + b` returns sum
`-`	Subtraction	`a - b` returns difference

Operator	Description	Example
`*`	Multiplication	`a * b` returns product
`/`	Division	`a / b` returns quotient
`**`	Exponentiation	`a ** b` returns a to power b
`%`	Modulus	`a % b` returns remainder
`//`	Floor Division	`a // b` returns quotient (integer)

Comparison Operators

Comparison operators evaluate conditions and return Boolean values (True or False). These are fundamental to conditional statements:

Operator	Description	Example
`==`	Equal to	`a == b`
`!=`	Not equal to	`a != b`
`>`	Greater than	`a > b`
`<`	Less than	`a < b`
`>=`	Greater than or equal	`a >= b`
`<=`	Less than or equal	`a <= b`

Logical Operators

Logical operators combine multiple conditions:

Operator	Description	Example
`and`	Returns True if both conditions are True	`a > 0 and b > 0`
`or`	Returns True if at least one condition is True	`a > 0 or b > 0`
`not`	Reverses the Boolean result	`not(a > 0)`

Practical Example:

```
age = 25
income = 50000

# Using logical operators
if age &gt;= 18 and income &gt;= 30000:
    print("Eligible for loan")

# Using or operator
if age &lt; 18 or income &lt; 10000:
    print("Not eligible")
```

```
# Using not operator
if not(age &lt; 21):
    print("Adult")
```

if, elif, and else Statements

The `if` statement executes code only if a condition is True. The `elif` (else if) statement allows checking multiple conditions, and `else` provides a default action:

```
score = 85

if score &gt;= 90:
    print("Grade: A")
elif score &gt;= 80:
    print("Grade: B")
elif score &gt;= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Key Points:

- Only the first true condition's block executes
- `elif` allows avoiding nested if statements
- `else` is optional and executes if all previous conditions are False
- Proper indentation (4 spaces) is mandatory

While Loops and Control Flow

A `while` loop executes a block of code as long as a condition is True. It's useful when you don't know how many iterations are needed:

```
count = 0
while count &lt; 5:
    print(f"Count: {count}")
    count += 1

# Infinite loop prevention - always ensure condition becomes False
user_input = ""
while user_input != "quit":
    user_input = input("Enter 'quit' to exit: ")
    if user_input != "quit":
        print(f"You entered: {user_input}")
```

Important: Always ensure your while loop condition eventually becomes False, or your program will hang indefinitely.

For Loops and Nested Loops

The `for` loop iterates over sequences (lists, tuples, strings, etc.):

```
# Simple for loop
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# For loop with range
for i in range(5):
    print(i)  # Output: 0, 1, 2, 3, 4

# Nested loops
for i in range(3):
    for j in range(3):
        print(f"({i}, {j})", end=" ")
    print()  # New line after inner loop

# Practical example: multiplication table
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i}*{j}={i*j}", end="  ")
    print()
```

For Loop vs While Loop:

- Use `for` when you know the number of iterations
- Use `while` when iteration depends on a condition

pass, break, and continue Statements

These statements modify loop behavior:

`break` - Exits the loop immediately:

```
for i in range(10):
    if i == 5:
        break
    print(i)  # Output: 0, 1, 2, 3, 4
```

`continue` - Skips the current iteration and moves to the next:

```
for i in range(5):
    if i == 2:
        continue
    print(i)  # Output: 0, 1, 3, 4
```

`pass` - Placeholder that does nothing (useful for empty code blocks):

```
for i in range(5):
    if i == 2:
        pass  # Placeholder for future code
    else:
        print(i)

# Also used in class definitions
class MyClass:
    pass  # Empty class for now
```

Nested Loops and List/Dictionary Comprehensions

Nested Loops were discussed earlier. **Comprehensions** provide a concise way to create lists and dictionaries:

List Comprehension:

```
# Traditional way
squares = []
for i in range(5):
    squares.append(i ** 2)

# Using list comprehension
squares = [i ** 2 for i in range(5)]
# Output: [0, 1, 4, 9, 16]

# With condition
even_squares = [i ** 2 for i in range(10) if i % 2 == 0]
# Output: [0, 4, 16, 36, 64]

# Nested list comprehension
matrix = [[i + j for j in range(3)] for i in range(3)]
# Output: [[0, 1, 2], [1, 2, 3], [2, 3, 4]]
```

Dictionary Comprehension:

```
# Traditional way
squares_dict = {}
for i in range(5):
    squares_dict[i] = i ** 2

# Using dictionary comprehension
squares_dict = {i: i ** 2 for i in range(5)}
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# From another dictionary
student_scores = {"Alice": 85, "Bob": 90, "Charlie": 78}
high_scores = {name: score for name, score in student_scores.items() if score &gt;= 85}
# Output: {'Alice': 85, 'Bob': 90}
```

Performance Note: List comprehensions are typically faster and more Pythonic than traditional loops when creating collections.

Part 2: Functions in Python

Understanding Functions

A **function** is a reusable block of code that performs a specific task. Functions promote code reusability, readability, and maintainability—critical skills for AI development.

Types of Functions

1. Built-in Functions

Python provides pre-defined functions like `print()`, `len()`, `sum()`, `max()`, etc.

2. User-Defined Functions

You create these using the `def` keyword:

```
def greet(name):
    """This function greets someone."""
    print(f"Hello, {name}!")

greet("Alice")  # Output: Hello, Alice!
```

Function Basics

```
def function_name(parameters):
    """Docstring explaining the function."""
    # Function body
    return result

# Calling the function
function_name(arguments)
```

Components:

- `def` keyword declares a function
- **Function name** should be descriptive and use snake_case
- **Parameters** are variables that receive values
- **Docstring** describes what the function does
- **Return statement** specifies what the function outputs

Code Optimization and Argument Functions

Default Arguments

```
def calculate_total(price, tax_rate=0.1):
    """Calculate total price including tax."""
    return price + (price * tax_rate)

total1 = calculate_total(100)        # Uses default tax_rate of 0.1
total2 = calculate_total(100, 0.15)  # Uses custom tax_rate
```

Keyword Arguments

```
def create_profile(name, age, city="Unknown"):
    """Create user profile."""
    return f"Name: {name}, Age: {age}, City: {city}"

# Called with keyword arguments
profile = create_profile(city="Delhi", name="Raj", age=25)
```

*args (Variable Positional Arguments)

```
def sum_numbers(*numbers):
    """Sum any number of arguments."""
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_numbers(1, 2, 3))          # Output: 6
print(sum_numbers(1, 2, 3, 4, 5))    # Output: 15
```

**kwargs (Variable Keyword Arguments)

```
def print_info(**kwargs):
    """Print key-value pairs."""
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25, city="Delhi")
# Output: name: Alice, age: 25, city: Delhi
```

Combining All Argument Types

```
def complex_function(a, b, *args, c=10, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}")
    print(f"c={c}")
    print(f"kwargs={kwargs}")

complex_function(1, 2, 3, 4, c=20, name="Alice", age=25)
```

Understanding Scope

Scope defines where a variable can be accessed. Python follows the LEGB rule:

Level	Description
Local	Inside functions
Enclosing	In nested functions
Global	At module level
Built-in	Python's built-in scope

```
global_var = 10  # Global scope

def outer_function():
    enclosing_var = 20  # Enclosing scope

    def inner_function():
        local_var = 30  # Local scope
        print(local_var)      # 30 (Local)
        print(enclosing_var)  # 20 (Enclosing)
        print(global_var)     # 10 (Global)

    inner_function()

outer_function()

# Modifying global variables
counter = 0

def increment():
    global counter
    counter += 1

increment()
print(counter)  # Output: 1
```

Lambda Functions

Lambda functions are anonymous, single-line functions. They're useful for simple operations:

```
# Traditional function
def add(x, y):
    return x + y

# Lambda equivalent
add_lambda = lambda x, y: x + y

print(add_lambda(5, 3))  # Output: 8

# Common use case: sorting
students = [("Alice", 85), ("Bob", 92), ("Charlie", 78)]
sorted_students = sorted(students, key=lambda x: x[^1])
# Output: [('Charlie', 78), ('Alice', 85), ('Bob', 92)]

# With filter
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
# Output: [2, 4, 6]
```

Map, Filter, and Reduce

These functional programming tools process iterables efficiently:

Map()

Applies a function to every item in an iterable:

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
# Output: [1, 4, 9, 16, 25]

# Real-world: converting temperatures
celsius = [0, 10, 20, 30]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))
# Output: [32.0, 50.0, 68.0, 86.0]
```

Filter()

Returns items that satisfy a condition:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
# Output: [2, 4, 6, 8, 10]

# Real-world: filtering valid data
scores = [45, 67, 89, 34, 92, 55]
```

```
passing_scores = list(filter(lambda x: x &gt;= 60, scores))
# Output: [67, 89, 92]
```

Reduce()

Reduces an iterable to a single value:

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
# Output: 120 (1*2*3*4*5)

# Real-world: calculating total bill
items = [100, 50, 75, 200]
total = reduce(lambda x, y: x + y, items)
# Output: 425
```

Part 3: Modules and Imports

What is a Module?

A **module** is a Python file containing functions, variables, and classes that can be imported and used in other programs. Python has a vast standard library of built-in modules.

Importing Modules

```
# Import entire module
import math
print(math.sqrt(16))  # Output: 4.0

# Import specific items
from math import sqrt, pi
print(sqrt(16))  # Output: 4.0
print(pi)        # Output: 3.14159...

# Import with alias
import numpy as np
from datetime import datetime as dt

# Import all items (generally discouraged)
from math import *
```

Using help() and dir()

`help()` provides documentation:

```
help(print)           # Shows print function documentation
help(math.sqrt)       # Shows sqrt function documentation

# Get help for a module
import os
help(os)
```

`dir()` lists available attributes and methods:

```
import math
print(dir(math))      # Lists all functions in math module
# Output: ['__doc__', '__name__', 'acos', 'acosh', ..., 'sqrt', ...]

# Check if function exists
if 'sqrt' in dir(math):
    print("sqrt function exists")
```

Important Python Modules

1. Math Module

Provides mathematical functions and constants:

```
import math

print(math.sqrt(16))      # 4.0
print(math.pow(2, 3))     # 8.0
print(math.ceil(4.3))     # 5 (round up)
print(math.floor(4.7))    # 4 (round down)
print(math.sin(math.pi/2)) # 1.0
print(math.log(10))       # Natural logarithm
print(math.pi)            # 3.14159...
print(math.e)             # 2.71828...
```

2. Random Module

Generates random numbers:

```
import random

print(random.random())           # Random float between 0 and 1
print(random.randint(1, 10))     # Random integer between 1 and 10
print(random.choice([1, 2, 3]))  # Random element from list
print(random.uniform(1, 10))     # Random float between 1 and 10

# Shuffle a list
```

```
items = [1, 2, 3, 4, 5]
random.shuffle(items)
print(items)  # Shuffled list

# Real-world: machine learning train-test split
data = list(range(100))
random.shuffle(data)
train = data[:80]
test = data[80:]
```

3. Datetime Module

Works with dates and times:

```
from datetime import datetime, date, timedelta

# Current date and time
now = datetime.now()
print(now)  # 2025-11-27 13:30:45.123456

# Current date only
today = date.today()
print(today)  # 2025-11-27

# Create specific date
specific_date = date(2024, 1, 15)

# Date arithmetic
tomorrow = today + timedelta(days=1)
next_week = today + timedelta(weeks=1)
next_month = today + timedelta(days=30)

# Format dates
print(now.strftime("%Y-%m-%d"))           # 2025-11-27
print(now.strftime("%d/%m/%Y"))           # 27/11/2025
print(now.strftime("%H:%M:%S"))           # 13:30:45
print(now.strftime("%A, %B %d, %Y"))      # Thursday, November 27, 2025
```

4. OS Module

Provides operating system interactions:

```
import os

# Get current working directory
print(os.getcwd())

# Change directory
os.chdir('/path/to/directory')

# List files in directory
files = os.listdir('.')
print(files)
```

```
# Check if file exists
if os.path.exists('filename.txt'):
    print("File exists")

# Create directory
os.makedirs('new_folder', exist_ok=True)

# File path operations
path = os.path.join('folder', 'subfolder', 'file.txt')
print(path)  # Handles platform-specific path separators

# Get file size
size = os.path.getsize('filename.txt')

# Remove file
os.remove('filename.txt')

# Get environment variables
home_dir = os.getenv('HOME')
```

Part 4: File Handling

Creating and Opening Files

```
# Create a new file
with open('myfile.txt', 'w') as file:
    file.write('Hello, World!')

# The 'with' statement automatically closes the file
```

File Opening Modes

Mode	Description
`'r'`	Read (default) - file must exist
`'w'`	Write - creates new file or overwrites existing
`'a'`	Append - adds to end of file
`'x'`	Create - fails if file exists
`'b'`	Binary mode (e.g., `'rb'`, `'wb'`)
`'+'`	Read and write (e.g., `'r+'`, `'w+'`)

Reading Files

```
# Read entire file
with open('myfile.txt', 'r') as file:
    content = file.read()
    print(content)

# Read line by line
with open('myfile.txt', 'r') as file:
    for line in file:
        print(line.strip())  # .strip() removes newline

# Read all lines as list
with open('myfile.txt', 'r') as file:
    lines = file.readlines()
    print(lines)

# Read specific number of characters
with open('myfile.txt', 'r') as file:
    first_50_chars = file.read(50)
```

Writing Files

```
# Write to file (overwrites)
with open('output.txt', 'w') as file:
    file.write('First line\n')
    file.write('Second line\n')

# Append to file
with open('output.txt', 'a') as file:
    file.write('Third line\n')

# Write multiple lines
lines = ['Line 1', 'Line 2', 'Line 3']
with open('output.txt', 'w') as file:
    file.writelines([line + '\n' for line in lines])
```

File Operations

```
# Check if file exists
import os
if os.path.exists('myfile.txt'):
    print("File exists")

# Get file size
size = os.path.getsize('myfile.txt')
print(f"File size: {size} bytes")

# Copy file
import shutil
shutil.copy('myfile.txt', 'myfile_backup.txt')
```

```
# Delete file
os.remove('myfile.txt')

# Rename file
os.rename('oldname.txt', 'newname.txt')
```

Errors and Exception Handling

Exceptions are errors that occur during program execution. Python allows you to handle them gracefully:

```
# Basic try-except
try:
    file = open('nonexistent.txt', 'r')
except FileNotFoundError:
    print("File not found!")

# Multiple exceptions
try:
    number = int("abc")
except FileNotFoundError:
    print("File not found")
except ValueError:
    print("Invalid value")

# Generic exception (catch-all)
try:
    result = 10 / 0
except Exception as e:
    print(f"An error occurred: {e}")

# Try-except-else-finally
try:
    file = open('myfile.txt', 'r')
    content = file.read()
except FileNotFoundError:
    print("File not found")
else:
    print("File read successfully")
finally:
    file.close()  # Always executes

# Better: using 'with' statement (automatically handles closing)
try:
    with open('myfile.txt', 'r') as file:
        content = file.read()
except FileNotFoundError:
    print("File not found")
```

Common Exceptions:

- `FileNotFoundError` - File doesn't exist
- `ValueError` - Invalid value for data type

- `TypeError` - Wrong data type
- `ZeroDivisionError` - Division by zero
- `IndexError` - Index out of range
- `KeyError` - Dictionary key doesn't exist

Part 5: Object-Oriented Programming (OOP)

Creating Classes

A **class** is a blueprint for creating objects. It defines attributes (data) and methods (functions):

```
class Person:
    """A simple Person class."""

    def __init__(self, name, age):
        """Initialize person with name and age."""
        self.name = name
        self.age = age

    def introduce(self):
        """Introduce the person."""
        print(f"Hello, I'm {self.name} and I'm {self.age} years old.")

# Create an object (instance)
person1 = Person("Alice", 25)
person1.introduce()  # Output: Hello, I'm Alice and I'm 25 years old.
```

Creating Objects

An **object** is an instance of a class:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

# Creating multiple objects
car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Civic")

print(car1.brand)  # Toyota
print(car2.model)  # Civic
```

The init() Method

The `__init__()` method is a **constructor** that initializes object attributes when an instance is created:

```
class Student:
    def __init__(self, name, student_id, gpa=0.0):
        self.name = name
        self.student_id = student_id
        self.gpa = gpa

student = Student("Bob", "S001", 3.8)
```

Object Methods

Methods are functions that operate on object data:

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        """Deposit money."""
        self.balance += amount
        print(f"Deposited: ${amount}")

    def withdraw(self, amount):
        """Withdraw money."""
        if amount &lt;= self.balance:
            self.balance -= amount
            print(f"Withdrawn: ${amount}")
        else:
            print("Insufficient balance")

    def get_balance(self):
        """Get current balance."""
        return self.balance

account = BankAccount("Alice", 1000)
account.deposit(500)              # Deposited: $500
account.withdraw(200)             # Withdrawn: $200
print(account.get_balance())       # 1300
```

Self Parameter

`self` represents the instance itself. It's the first parameter in all instance methods:

```
class Counter:
    def __init__(self):
        self.count = 0

    def increment(self):
```

```
        self.count += 1  # self refers to the specific instance

    def get_count(self):
        return self.count

counter1 = Counter()
counter2 = Counter()

counter1.increment()
counter1.increment()

print(counter1.get_count())  # 2
print(counter2.get_count())  # 0 (separate instance)
```

Modifying Objects

Objects can be modified by changing their attributes or calling methods:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

employee = Employee("Charlie", 50000)

# Direct modification
employee.salary = 55000

# Or through a method
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def give_raise(self, amount):
        self.salary += amount

employee.give_raise(5000)
print(employee.salary)  # 55000
```

Deleting Objects and Attributes

```
class Person:
    def __init__(self, name):
        self.name = name

person = Person("Alice")

# Delete attribute
del person.name

# Delete entire object
del person
```

```
# Trying to access deleted object will raise error
# print(person)  # NameError: name 'person' is not defined
```

Class and Instance Attributes

```
class Dog:
    species = "Canis familiaris"  # Class attribute (shared by all instances)

    def __init__(self, name, age):
        self.name = name           # Instance attribute
        self.age = age

dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

print(dog1.species)  # Canis familiaris
print(dog1.name)     # Buddy
print(dog2.name)     # Max

# All instances share class attribute
print(Dog.species)   # Canis familiaris
```

Inheritance

Inheritance allows creating new classes based on existing ones:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):  # Dog inherits from Animal
    def speak(self):
        return f"{self.name} barks!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows!"

dog = Dog("Buddy")
cat = Cat("Whiskers")

print(dog.speak())    # Buddy barks!
print(cat.speak())    # Whiskers meows!

# Using super() to access parent class methods
class Dog(Animal):
    def __init__(self, name, breed):
```

```
        super().__init__(name)  # Call parent __init__
        self.breed = breed
```

Polymorphism and Method Overriding

Polymorphism allows objects of different types to be treated uniformly:

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(shape.area())  # Calls appropriate area() method
```

References and Further Learning

1. van Rossum, G., Warsaw, B., & Coghlan, A. (2024). PEP 8 – Style Guide for Python Code. Python.org.
2. Real Python. (2024). Object-Oriented Programming (OOP) in Python. Retrieved from https://realpython.com/
3. GeeksforGeeks. (2024). Python Tutorial – Complete Guide for Beginners. Retrieved from https://www.geeksforgeeks.org/
4. Python Software Foundation. (2024). Python 3.12 Official Documentation. Retrieved from https://docs.python.org/3/
5. Sweigart, A. (2023). Automate the Boring Stuff with Python: Practical Programming for Total Beginners (2nd ed.). No Starch Press.
6. Ramalho, L. (2022). Fluent Python: Clear, Concise, and Effective Programming (2nd ed.). O'Reilly Media.
7. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2023). Data Structures and Algorithms in Python. Wiley.

8. Learish, K., & Patel, D. (2024). Functional Programming in Python. DataCamp. Retrieved from https://www.datacamp.com/
9. StackOverflow Community. (2024). Python Exception Handling Best Practices. Retrieved from https://stackoverflow.com/
10. Hilpisch, Y. (2023). Python for Finance: Mastering Data-Driven Finance (2nd ed.). O'Reilly Media.

Key Takeaways for Your AI Career

- **Control flow** structures are essential for building decision-making algorithms and data processing pipelines
- **Functions** with proper scope management enable scalable code that's maintainable and reusable
- **Modules** let you leverage Python's powerful ecosystem for ML and data science
- **File handling** with proper exception handling ensures your data processing is robust
- **OOP principles** help structure complex ML systems with classes for models, preprocessors, and pipelines

As you progress in your AI/ML career, these fundamentals will serve as the foundation for frameworks like scikit-learn, TensorFlow, and PyTorch. Master them well!
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19]

⁂

1. https://blog.ksrdatavision.com/python/flow-controls-in-python/
2. https://www.youtube.com/watch?v=HsfiWodjhrg
3. https://www.geeksforgeeks.org/python/python-datetime-module/
4. https://www.thefullstack.co.in/python-modules-and-packages-guide/
5. https://apxml.com/courses/python-for-beginners/chapter-7-reusing-code-modules-packages/python-common-modules
6. https://www.scribd.com/document/919771434/Popular-Modules-Random-Math-Datetime-Os
7. https://www.almabetter.com/bytes/tutorials/python/python-inheritance-and-polymorphism
8. https://www.learnpython.org/en/Map,_Filter,_Reduce
9. https://www.scholarhat.com/tutorial/python/modules-in-python
10. https://www.datacamp.com/tutorial/python-inheritance
11. https://www.projectguru.in/controlling-the-flow-of-execution-in-python-programming/
12. https://www.geeksforgeeks.org/python/map-reduce-and-filter-operations-in-python/
13. https://pynative.com/python-control-flow-statements/
14. https://www.alooba.com/skills/concepts/python-16/control-flow/
15. https://realpython.com/python-control-flow/
16. https://www.kdnuggets.com/functional-programming-in-python-leveraging-lambda-functions-and-higher-order-functions

17. https://stackoverflow.com/questions/5627425/what-is-a-good-way-to-handle-exceptions-when-trying-to-read-a-file-in-python
18. https://realpython.com/python3-object-oriented-programming/
19. https://stackoverflow.com/questions/77922414/python-lists-why-is-the-pythonic-list-comprehension-8-20-slower